



Concentration Bounds in Bandit Analysis

Hoeffding, Bernstein, and Sub-Gaussian Tails

Author: Tang

Institute: Tang's Machine Learning Blog

Date: 2026-06-18

Version: 1.0.0

Topic: Concentration Bounds, UCB, Bandit Analysis

Contents

Chapter 1 The Error Bar Is the Algorithm	1
1.1 A small picture	1
Chapter 2 One Random Number, Then Many	2
2.1 The event we care about	2
Chapter 3 Hoeffding: The First Error Bar	3
3.1 The proof idea in one line	3
3.2 Step-by-step proof	3
Chapter 4 From Tail Bound to Confidence Radius	5
Chapter 5 Why Bandits Need Many Error Bars at Once	6
Chapter 6 UCB from Hoeffding	7
6.1 Algorithm design	7
6.2 The standard pull-count calculation	7
Chapter 7 Bernstein: When Variance Matters	9
7.1 How to read the formula	9
7.2 Empirical Bernstein UCB	10
Chapter 8 Sub-Gaussian Tails: A Cleaner Language	11
8.1 Tail bound from the definition	11
Chapter 9 A Small Experiment	13
9.1 Fixed-time coverage	13
9.2 Bandit results	14
Chapter 10 Complete Experiment Code	16
Chapter 11 What to Remember	18
Appendix A. Formula Sheet	19
Appendix B. Notation Table	20
Appendix C. Full Experiment Script	21
References	26

Chapter 1 The Error Bar Is the Algorithm

A bandit algorithm looks like a decision rule. It chooses an arm. It receives a reward. It chooses again. But inside almost every clean bandit proof there is a smaller object doing most of the work: an error bar.

If the empirical mean of an arm is the number we can see, the true mean is the number we want. The distance between them is invisible. Concentration inequalities are the tools that make this invisible distance usable.

Key idea

A concentration bound is not just a probability lemma. In bandit theory it becomes an algorithmic object. It tells the learner how much optimism is still defensible after seeing a certain amount of data.

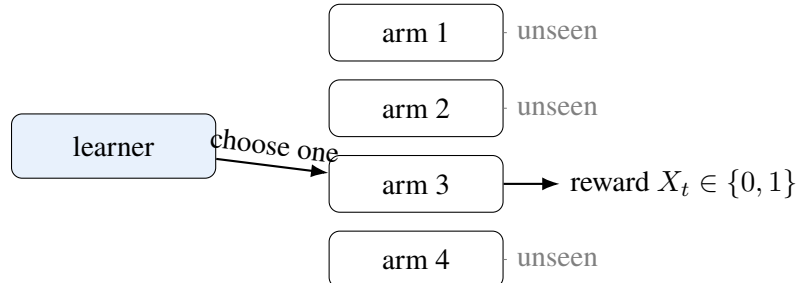
The whole story of UCB can be read as one sentence:

choose the arm with the largest plausible mean.

The word “plausible” is where probability enters.

1.1 A small picture

Consider four actions. Each has an unknown click probability. The learner sees only the reward of the action it chose.



The hard part is not that rewards are random. The hard part is that the data are produced by our past decisions. If an arm looks bad early, we may stop sampling it. If it looks good early, we may sample it more. The sample size of each arm is itself a random outcome of the algorithm.

This is why concentration in bandits must be slightly more careful than concentration in a fixed dataset.

Chapter 2 One Random Number, Then Many

We start from the smallest possible object.

A random variable is a number we do not know yet. If X is a Bernoulli reward, then

$$X = \begin{cases} 1, & \text{click or success,} \\ 0, & \text{no click or failure.} \end{cases}$$

Its mean is

$$\mu = \mathbb{E}[X].$$

For a Bernoulli random variable with success probability p ,

$$\mathbb{E}[X] = 1 \cdot p + 0 \cdot (1 - p) = p.$$

Now suppose we pull the same arm n times and obtain

$$X_1, X_2, \dots, X_n.$$

The empirical mean is

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n X_i.$$

It is the most natural estimate of μ because

$$\mathbb{E}[\hat{\mu}_n] = \mathbb{E}\left[\frac{1}{n} \sum_{i=1}^n X_i\right] = \frac{1}{n} \sum_{i=1}^n \mathbb{E}[X_i] = \frac{1}{n} \cdot n\mu = \mu.$$

Think

The sample mean is unbiased, but unbiased does not mean correct. It means it is right on average over repeated worlds. In one actual run, it can still be too high or too low. Concentration asks: by how much?

2.1 The event we care about

The basic bad event is

$$\{\hat{\mu}_n - \mu \geq r\}.$$

This event says: the empirical mean is too optimistic by at least r .

The two-sided bad event is

$$\{|\hat{\mu}_n - \mu| \geq r\} = \{\hat{\mu}_n - \mu \geq r\} \cup \{\mu - \hat{\mu}_n \geq r\}.$$

Therefore

$$\mathbb{P}(|\hat{\mu}_n - \mu| \geq r) \leq \mathbb{P}(\hat{\mu}_n - \mu \geq r) + \mathbb{P}(\mu - \hat{\mu}_n \geq r).$$

This is the union bound. It says that if either of two things can go wrong, the chance that something goes wrong is at most the sum of the two chances.

Chapter 3 Hoeffding: The First Error Bar

Hoeffding's inequality is the first concentration bound one should learn for bandits. It is simple, robust, and often good enough.

Result

Let $X_1, \dots, X_n$ be independent random variables in $[0, 1]$ with common mean μ . Then for every $r > 0$,

$$\mathbb{P}(\hat{\mu}_n - \mu \geq r) \leq \exp(-2nr^2),$$

and

$$\mathbb{P}(|\hat{\mu}_n - \mu| \geq r) \leq 2 \exp(-2nr^2).$$

The statement says that the probability of a large error decays exponentially in nr^2 . More data makes the error smaller. A larger error is less likely.

3.1 The proof idea in one line

A probability of a large deviation is hard to bound directly. Hoeffding's proof turns the deviation into an exponential moment:

$$\mathbb{P}(S_n \geq nr) \longrightarrow \mathbb{E}[\exp(\lambda S_n)],$$

where

$$S_n = \sum_{i=1}^n (X_i - \mu).$$

The exponential function is useful because products appear when independent variables are added.

3.2 Step-by-step proof

Let

$$S_n = \sum_{i=1}^n (X_i - \mu).$$

Then

$$\hat{\mu}_n - \mu = \frac{1}{n} \sum_{i=1}^n (X_i - \mu) = \frac{S_n}{n}.$$

Hence

$$\{\hat{\mu}_n - \mu \geq r\} = \{S_n \geq nr\}.$$

For any $\lambda > 0$,

$$S_n \geq nr \iff \exp(\lambda S_n) \geq \exp(\lambda nr).$$

By Markov's inequality,

$$\mathbb{P}(\exp(\lambda S_n) \geq \exp(\lambda nr)) \leq \frac{\mathbb{E}[\exp(\lambda S_n)]}{\exp(\lambda nr)}.$$

Therefore

$$\mathbb{P}(S_n \geq nr) \leq \exp(-\lambda nr) \mathbb{E}[\exp(\lambda S_n)].$$

Now expand S_n :

$$\mathbb{E}[\exp(\lambda S_n)] = \mathbb{E} \left[\exp \left(\lambda \sum_{i=1}^n (X_i - \mu) \right) \right].$$

Since the exponential of a sum is a product,

$$= \mathbb{E} \left[\prod_{i=1}^n \exp(\lambda(X_i - \mu)) \right].$$

Since X_i are independent,

$$= \prod_{i=1}^n \mathbb{E}[\exp(\lambda(X_i - \mu))].$$

Hoeffding's lemma for a centered random variable in an interval of length 1 gives

$$\mathbb{E}[\exp(\lambda(X_i - \mu))] \leq \exp\left(\frac{\lambda^2}{8}\right).$$

Thus

$$\mathbb{E}[\exp(\lambda S_n)] \leq \prod_{i=1}^n \exp\left(\frac{\lambda^2}{8}\right) = \exp\left(\frac{n\lambda^2}{8}\right).$$

Putting this back into the probability bound,

$$\mathbb{P}(S_n \geq nr) \leq \exp(-\lambda nr) \exp\left(\frac{n\lambda^2}{8}\right) = \exp\left(-\lambda nr + \frac{n\lambda^2}{8}\right).$$

This is true for every $\lambda > 0$. We choose the best λ .

Let

$$\phi(\lambda) = -\lambda nr + \frac{n\lambda^2}{8}.$$

Then

$$\phi'(\lambda) = -nr + \frac{n\lambda}{4}.$$

Set $\phi'(\lambda) = 0$:

$$-nr + \frac{n\lambda}{4} = 0 \implies \lambda = 4r.$$

Substitute $\lambda = 4r$:

$$\phi(4r) = -(4r)nr + \frac{n(4r)^2}{8} = -4nr^2 + \frac{16nr^2}{8} = -4nr^2 + 2nr^2 = -2nr^2.$$

Therefore

$$\mathbb{P}(\hat{\mu}_n - \mu \geq r) = \mathbb{P}(S_n \geq nr) \leq \exp(-2nr^2).$$

The same argument applied to $-S_n$ gives

$$\mathbb{P}(\mu - \hat{\mu}_n \geq r) \leq \exp(-2nr^2).$$

Finally,

$$\mathbb{P}(|\hat{\mu}_n - \mu| \geq r) \leq \exp(-2nr^2) + \exp(-2nr^2) = 2\exp(-2nr^2).$$

Key idea

Hoeffding turns “the sample mean might be wrong” into a number. That number is an error bar.

Chapter 4 From Tail Bound to Confidence Radius

The inequality

$$\mathbb{P}(|\hat{\mu}_n - \mu| \geq r) \leq 2 \exp(-2nr^2)$$

is not yet an algorithm. An algorithm needs a radius r .

Choose a failure probability $\delta \in (0, 1)$. We want

$$2 \exp(-2nr^2) \leq \delta.$$

Now solve for r step by step:

$$2 \exp(-2nr^2) \leq \delta,$$

$$\exp(-2nr^2) \leq \frac{\delta}{2},$$

$$-2nr^2 \leq \log\left(\frac{\delta}{2}\right),$$

$$2nr^2 \geq \log\left(\frac{2}{\delta}\right),$$

$$r^2 \geq \frac{\log(2/\delta)}{2n},$$

$$r \geq \sqrt{\frac{\log(2/\delta)}{2n}}.$$

Therefore the Hoeffding confidence radius is

$$\text{rad}_H(n, \delta) = \sqrt{\frac{\log(2/\delta)}{2n}}.$$

With probability at least $1 - \delta$,

$$\mu \in [\hat{\mu}_n - \text{rad}_H(n, \delta), \hat{\mu}_n + \text{rad}_H(n, \delta)].$$

Think

Do not remember only the formula. Remember the tradeoff: the radius shrinks like $1/\sqrt{n}$ and grows like $\sqrt{\log(1/\delta)}$. Confidence is not free. Asking for a very small failure probability makes the interval wider.

Chapter 5 Why Bandits Need Many Error Bars at Once

In supervised learning, one often analyzes a fixed estimator at a fixed sample size. In bandits, the learner keeps looking, choosing, and updating. At time t , each arm a has its own random number of pulls

$$N_a(t) = \sum_{s=1}^t \mathbf{1}\{A_s = a\}.$$

The sample mean of arm a is

$$\hat{\mu}_a(t) = \frac{1}{N_a(t)} \sum_{s=1}^t \mathbf{1}\{A_s = a\} X_s,$$

when $N_a(t) > 0$.

The learner wants all confidence intervals to be correct at all relevant times. A simple way to do this is to buy many error bars with one failure budget.

Suppose there are K arms and T rounds. There are at most KT arm-time pairs. Give each pair failure probability

$$\delta' = \frac{\delta}{KT}.$$

For one pair (a, t) , Hoeffding gives

$$\mathbb{P} \left(|\hat{\mu}_a(t) - \mu_a| > \sqrt{\frac{\log(2/\delta')}{2N_a(t)}} \right) \leq \delta'.$$

By the union bound,

$$\mathbb{P}(\text{at least one arm-time interval fails}) \leq \sum_{a=1}^K \sum_{t=1}^T \delta' = KT \cdot \frac{\delta}{KT} = \delta.$$

Thus, with probability at least $1 - \delta$, every displayed error bar is correct.

Key idea

The union bound is not a crude technicality here. It is the mechanism that lets an adaptive algorithm look at many arms many times without pretending that each look is isolated.

Chapter 6 UCB from Hoeffding

For arm a , define the optimistic index

$$U_a(t) = \hat{\mu}_a(t) + \sqrt{\frac{2 \log t}{N_a(t)}}.$$

The exact constant is not sacred. The structure is sacred:

index = what we have seen + how wrong it could still be.

At each round,

$$A_t \in \arg \max_{a \in [K]} U_a(t).$$

6.1 Algorithm design

Hoeffding-UCB core rule.

```
1 def ucb_hoeffding_action(t, sums, counts):
2     means = sums / counts
3     bonus = np.sqrt(2.0 * np.log(t + 1.0) / counts)
4     return int(np.argmax(means + bonus))
```

The code is short because the theorem is doing the work. The square-root term is the confidence radius.

6.2 The standard pull-count calculation

Let a^* be an optimal arm and let

$$\Delta_a = \mu_{a^*} - \mu_a > 0$$

for a suboptimal arm a .

On a good event, all confidence intervals are correct. For every arm b and time t ,

$$|\hat{\mu}_b(t) - \mu_b| \leq \sqrt{\frac{2 \log t}{N_b(t)}}.$$

If UCB chooses a suboptimal arm a at time t , then

$$U_a(t) \geq U_{a^*}(t).$$

Since the optimal arm's index is optimistic,

$$U_{a^*}(t) = \hat{\mu}_{a^*}(t) + \sqrt{\frac{2 \log t}{N_{a^*}(t)}} \geq \mu_{a^*}.$$

Therefore

$$U_a(t) \geq \mu_{a^*}.$$

But on the good event,

$$\hat{\mu}_a(t) \leq \mu_a + \sqrt{\frac{2 \log t}{N_a(t)}}.$$

Hence

$$U_a(t) = \hat{\mu}_a(t) + \sqrt{\frac{2 \log t}{N_a(t)}} \leq \mu_a + 2\sqrt{\frac{2 \log t}{N_a(t)}}.$$

Combining the lower and upper bounds on $U_a(t)$,

$$\mu_{a^*} \leq \mu_a + 2\sqrt{\frac{2 \log t}{N_a(t)}}.$$

Subtract μ_a :

$$\Delta_a \leq 2\sqrt{\frac{2 \log t}{N_a(t)}}.$$

Divide by 2:

$$\frac{\Delta_a}{2} \leq \sqrt{\frac{2 \log t}{N_a(t)}}.$$

Square both sides:

$$\frac{\Delta_a^2}{4} \leq \frac{2 \log t}{N_a(t)}.$$

Rearrange:

$$N_a(t) \leq \frac{8 \log t}{\Delta_a^2}.$$

Thus a suboptimal arm can be pulled many times only if its uncertainty is still large. Once it has been sampled enough, optimism can no longer hide its gap.

The regret contribution of arm a is approximately

$$\Delta_a \mathbb{E}[N_a(T)].$$

The calculation above explains the familiar logarithmic shape:

$$\Delta_a N_a(T) \lesssim \Delta_a \cdot \frac{8 \log T}{\Delta_a^2} = \frac{8 \log T}{\Delta_a}.$$

Chapter 7 Bernstein: When Variance Matters

Hoeffding only uses the fact that rewards lie in $[0, 1]$. It does not ask whether the arm is noisy or nearly deterministic.

But this matters. A Bernoulli arm with mean 0.5 has variance

$$0.5(1 - 0.5) = 0.25.$$

A Bernoulli arm with mean 0.05 has variance

$$0.05(1 - 0.05) = 0.0475.$$

Both rewards lie in $[0, 1]$, but the second one fluctuates much less.

Bernstein-type bounds use this variance information.

Result

A simple Bernstein-style confidence radius has the form

$$\text{rad}_B(n, \delta, \sigma^2) = \sqrt{\frac{2\sigma^2 \log(2/\delta)}{n}} + \frac{\log(2/\delta)}{3n}.$$

The first term is variance-sensitive. The second term is a boundedness correction.

7.1 How to read the formula

Compare Hoeffding and Bernstein:

$$\begin{aligned}\text{rad}_H(n, \delta) &= \sqrt{\frac{\log(2/\delta)}{2n}}, \\ \text{rad}_B(n, \delta, \sigma^2) &= \sqrt{\frac{2\sigma^2 \log(2/\delta)}{n}} + \frac{\log(2/\delta)}{3n}.\end{aligned}$$

If σ^2 is small, the square-root term is small. For large n , the $1/n$ correction becomes smaller than the $1/\sqrt{n}$ term, and the variance advantage becomes visible.

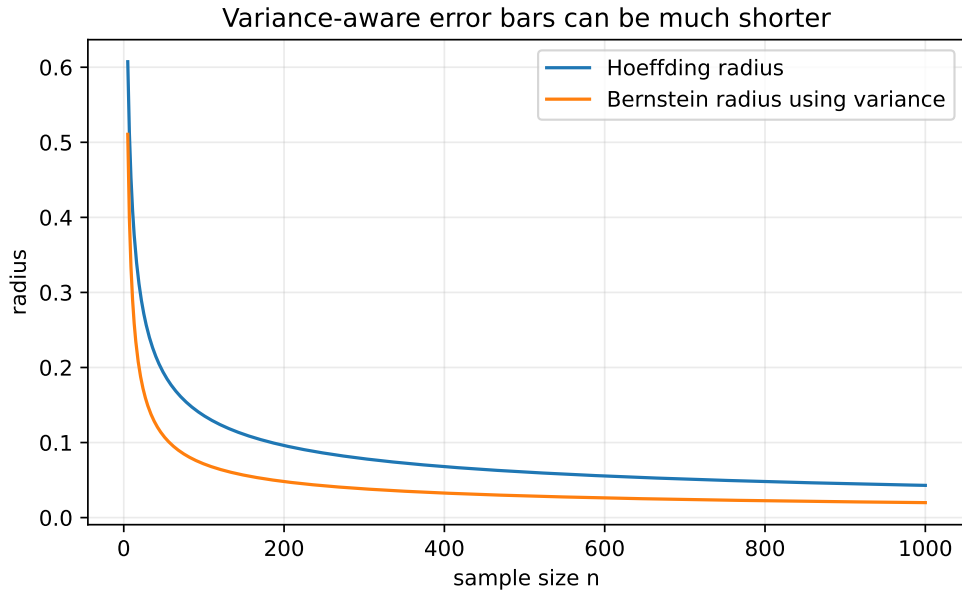


Figure 7.1: For a low-variance Bernoulli arm with $p = 0.05$, a variance-aware radius can be substantially shorter than Hoeffding’s radius.

7.2 Empirical Bernstein UCB

In a bandit problem, σ_a^2 is usually unknown. We estimate it.

For arm a , define

$$\hat{\sigma}_a^2(t) = \frac{1}{N_a(t)} \sum_{s:A_s=a} (X_s - \hat{\mu}_a(t))^2.$$

Then an empirical Bernstein-style index is

$$U_a^{\text{EB}}(t) = \hat{\mu}_a(t) + \sqrt{\frac{2\hat{\sigma}_a^2(t) \log t}{N_a(t)}} + \frac{3 \log t}{N_a(t)}.$$

Empirical-Bernstein UCB core rule.

```

1 def ucb_bernstein_action(t, sums, sq_sums, counts):
2     means = sums / counts
3     variances = np.maximum(0.0, sq_sums / counts - means * means)
4     log_term = np.log(t + 1.0)
5     bonus = np.sqrt(2.0 * variances * log_term / counts) + 3.0 * log_term / counts
6     return int(np.argmax(means + bonus))

```

This is the same design pattern as UCB-Hoeffding, but the error bar now listens to the observed variance.

Chapter 8 Sub-Gaussian Tails: A Cleaner Language

Hoeffding is excellent for bounded variables. But many models are not naturally bounded. A Gaussian reward, for example, can take any real value.

A random variable X with mean μ is called σ -sub-Gaussian if, for every $\lambda \in \mathbb{R}$,

$$\mathbb{E}[\exp(\lambda(X - \mu))] \leq \exp\left(\frac{\lambda^2 \sigma^2}{2}\right).$$

This definition says: the exponential moments of $X - \mu$ are no larger than those of a Gaussian with variance proxy σ^2 .

8.1 Tail bound from the definition

Let $X_1, \dots, X_n$ be independent σ -sub-Gaussian random variables with mean μ . Let

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n X_i.$$

We derive a one-sided tail bound.

Set

$$S_n = \sum_{i=1}^n (X_i - \mu).$$

Then

$$\hat{\mu}_n - \mu = \frac{S_n}{n}.$$

For $\lambda > 0$,

$$\begin{aligned} \mathbb{P}(\hat{\mu}_n - \mu \geq r) &= \mathbb{P}(S_n \geq nr) \\ &= \mathbb{P}(\exp(\lambda S_n) \geq \exp(\lambda nr)). \end{aligned}$$

By Markov's inequality,

$$\mathbb{P}(S_n \geq nr) \leq \exp(-\lambda nr) \mathbb{E}[\exp(\lambda S_n)].$$

By independence,

$$\mathbb{E}[\exp(\lambda S_n)] = \prod_{i=1}^n \mathbb{E}[\exp(\lambda(X_i - \mu))].$$

By sub-Gaussianity,

$$\leq \prod_{i=1}^n \exp\left(\frac{\lambda^2 \sigma^2}{2}\right) = \exp\left(\frac{n \lambda^2 \sigma^2}{2}\right).$$

Therefore

$$\mathbb{P}(\hat{\mu}_n - \mu \geq r) \leq \exp\left(-\lambda nr + \frac{n \lambda^2 \sigma^2}{2}\right).$$

Choose the best λ .

Let

$$\psi(\lambda) = -\lambda nr + \frac{n \lambda^2 \sigma^2}{2}.$$

Then

$$\psi'(\lambda) = -nr + n\lambda\sigma^2.$$

Set $\psi'(\lambda) = 0$:

$$-nr + n\lambda\sigma^2 = 0 \implies \lambda = \frac{r}{\sigma^2}.$$

Substitute:

$$\psi\left(\frac{r}{\sigma^2}\right) = -\frac{r}{\sigma^2}nr + \frac{n}{2}\frac{r^2}{\sigma^4}\sigma^2 = -\frac{nr^2}{\sigma^2} + \frac{nr^2}{2\sigma^2} = -\frac{nr^2}{2\sigma^2}.$$

Thus

$$\mathbb{P}(\hat{\mu}_n - \mu \geq r) \leq \exp\left(-\frac{nr^2}{2\sigma^2}\right).$$

The two-sided version is

$$\mathbb{P}(|\hat{\mu}_n - \mu| \geq r) \leq 2 \exp\left(-\frac{nr^2}{2\sigma^2}\right).$$

Solving

$$2 \exp\left(-\frac{nr^2}{2\sigma^2}\right) \leq \delta$$

gives

$$\text{rad}_{\text{SG}}(n, \delta, \sigma) = \sigma \sqrt{\frac{2 \log(2/\delta)}{n}}.$$

Key idea

Sub-Gaussianity is the cleanest language for many bandit papers. It says that averages concentrate like Gaussian averages, even when the rewards are not exactly Gaussian.

Chapter 9 A Small Experiment

We now run a minimal experiment. The environment has four Bernoulli arms with means

$$(0.01, 0.03, 0.05, 0.07).$$

The best arm is only slightly better than the others, and all rewards are low-variance because clicks are rare.

We compare four policies:

Greedy, UCB-Hoeffding, UCB-Bernstein, Thompson sampling.

The point is not to crown a universal winner. The point is to see how different uses of uncertainty change behavior.

9.1 Fixed-time coverage

First, we verify the simplest message: Hoeffding intervals are conservative at a fixed time.

Table 9.1: Empirical miss probability for Hoeffding intervals with nominal $\delta = 0.05$. The true Bernoulli mean is $p = 0.30$, and each row uses 50,000 Monte Carlo repetitions.

n	Hoeffding radius	Empirical miss probability
10	0.4295	0.00164
20	0.3037	0.00134
50	0.1921	0.00326
100	0.1358	0.00338
200	0.0960	0.00260
500	0.0607	0.00290
1000	0.0429	0.00358

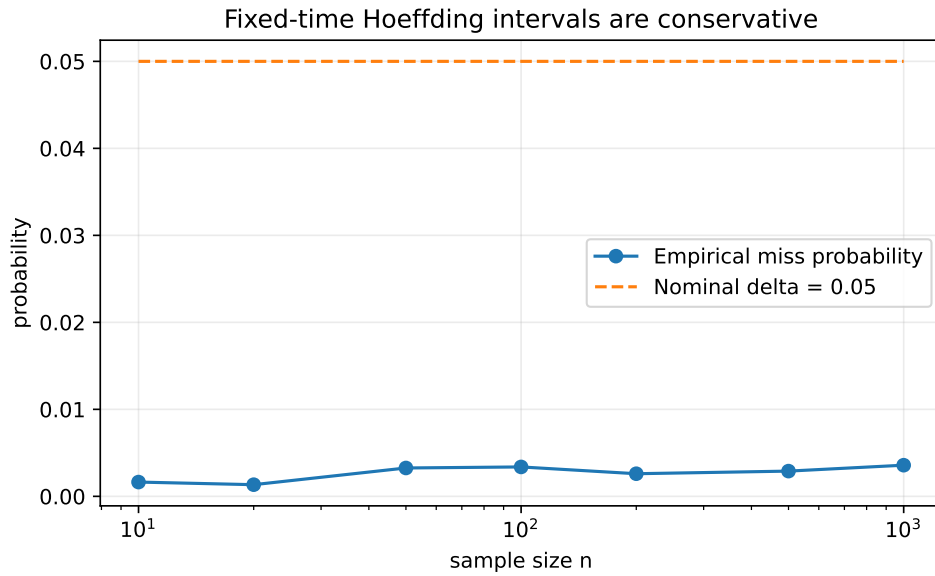


Figure 9.1: The empirical probability of missing the true mean stays below the nominal failure level. The bound is deliberately safe.

9.2 Bandit results

The following table reports average final regret over 150 independent runs and $T = 3000$ rounds.

Table 9.2: Average final regret and average number of pulls per arm.

Policy	Final regret	Arm 0	Arm 1	Arm 2	Arm 3
Greedy	157.51	2557.59	61.11	80.71	300.60
UCB-Hoeffding	70.64	492.59	627.50	799.19	1080.73
UCB-Bernstein	51.74	291.75	475.73	760.27	1472.25
Thompson	22.22	97.34	176.99	465.01	2260.65

Greedy often gets trapped by early noise. UCB-Hoeffding keeps exploring because the error bars are wide. UCB-Bernstein uses the low observed variance to reduce unnecessary exploration. Thompson sampling, although not a concentration-index algorithm, also converts uncertainty into action probabilities and quickly allocates more mass to the best arm.

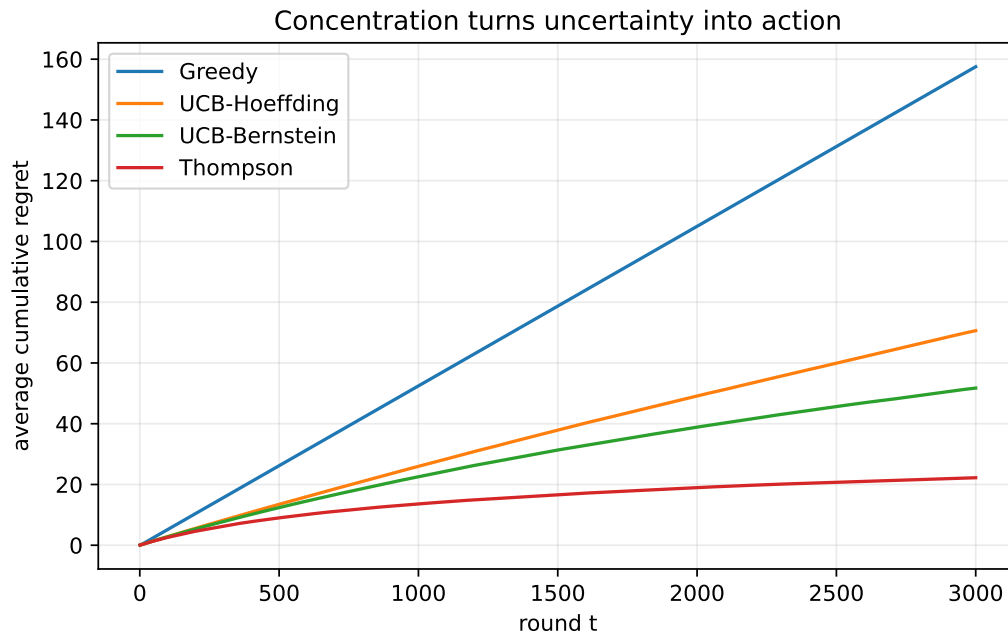


Figure 9.2: Average cumulative regret over 150 runs. Concentration controls how long uncertainty can justify exploration.

Chapter 10 Complete Experiment Code

The code below is the core of the simulation. The full script is included with this note.

Core implementation used in the experiment.

```
1 import numpy as np
2
3 def run_bandit(policy, p, T, rng):
4     K = len(p)
5     counts = np.zeros(K, dtype=int)
6     sums = np.zeros(K, dtype=float)
7     sq_sums = np.zeros(K, dtype=float)
8     regret = np.zeros(T, dtype=float)
9     p_star = float(np.max(p))
10    cumulative_regret = 0.0
11
12    # Pull each arm once, so every empirical mean is defined.
13    for t in range(min(K, T)):
14        a = t
15        r = rng.binomial(1, p[a])
16        counts[a] += 1
17        sums[a] += r
18        sq_sums[a] += r * r
19        cumulative_regret += p_star - p[a]
20        regret[t] = cumulative_regret
21
22    for t in range(K, T):
23        means = sums / np.maximum(counts, 1)
24
25        if policy == "Greedy":
26            a = int(np.argmax(means))
27
28        elif policy == "UCB-Hoeffding":
29            bonus = np.sqrt(2.0 * np.log(t + 1.0) / counts)
30            a = int(np.argmax(means + bonus))
31
32        elif policy == "UCB-Bernstein":
33            variances = np.maximum(0.0, sq_sums / counts - means * means)
34            log_term = np.log(t + 1.0)
35            bonus = np.sqrt(2.0 * variances * log_term / counts) \
36                    + 3.0 * log_term / counts
37            a = int(np.argmax(means + bonus))
38
39        elif policy == "Thompson":
40            alpha = sums + 1.0
41            beta = counts - sums + 1.0
42            a = int(np.argmax(rng.beta(alpha, beta)))
```

```
43
44     r = rng.binomial(1, p[a])
45     counts[a] += 1
46     sums[a] += r
47     sq_sums[a] += r * r
48     cumulative_regret += p_star - p[a]
49     regret[t] = cumulative_regret
50
51     return regret, counts
```

Chapter 11 What to Remember

Concentration bounds are often introduced as probability facts. In bandit theory they become operational objects.

- Hoeffding gives a simple radius when rewards are bounded.
- Bernstein improves the radius when variance is small.
- Sub-Gaussian tails provide a clean language for unbounded but well-behaved noise.
- UCB is just empirical performance plus an error bar.
- The regret proof works because a suboptimal arm cannot remain plausibly optimal after enough samples.

The deepest point is not the square root. It is the conversion:

probability $\longrightarrow$ confidence $\longrightarrow$ action $\longrightarrow$ regret control.

Appendix A. Formula Sheet

Table 11.1: Core formulas.

Object	Formula
Sample mean	$\hat{\mu}_n = n^{-1} \sum_{i=1}^n X_i$
Hoeffding tail	$\mathbb{P}(\hat{\mu}_n - \mu \geq r) \leq 2e^{-2nr^2}$
Hoeffding radius	$\sqrt{\log(2/\delta)/(2n)}$
Sub-Gaussian MGF	$\mathbb{E}[e^{\lambda(X-\mu)}] \leq e^{\lambda^2 \sigma^2/2}$
Sub-Gaussian radius	$\sigma \sqrt{2 \log(2/\delta)/n}$
Bernstein radius	$\sqrt{2\sigma^2 \log(2/\delta)/n} + \log(2/\delta)/(3n)$
Hoeffding-UCB index	$\hat{\mu}_a(t) + \sqrt{2 \log t / N_a(t)}$
Empirical-Bernstein index	$\hat{\mu}_a(t) + \sqrt{2\hat{\sigma}_a^2(t) \log t / N_a(t)} + 3 \log t / N_a(t)$

Appendix B. Notation Table

Table 11.2: Notation.

Symbol	Meaning
X_i	reward sample
μ	true mean of a reward distribution
$\hat{\mu}_n$	empirical mean after n samples
δ	allowed failure probability
r	confidence radius
K	number of arms
T	time horizon
A_t	arm chosen at time t
$N_a(t)$	number of times arm a has been pulled by time t
μ_a	true mean of arm a
a^*	an optimal arm
Δ_a	gap $\mu_{a^*} - \mu_a$
$U_a(t)$	optimistic index for arm a at time t

Appendix C. Full Experiment Script

Listing 11.1: Full experiment script.

```
1  """Experiments for the lecture note
2  Concentration Bounds in Bandit Analysis.
3
4  The script generates three figures and one CSV file:
5      1. fixed_time_hoeffding.pdf/png
6      2. confidence_radius_widths.pdf/png
7      3. ucb_regret_curves.pdf/png
8      4. concentration_results.csv
9
10 Only numpy, pandas, and matplotlib are used.
11 """
12
13 from __future__ import annotations
14
15 import math
16 from pathlib import Path
17
18 import numpy as np
19 import pandas as pd
20 import matplotlib.pyplot as plt
21
22 OUT = Path(__file__).resolve().parent
23
24
25 def hoeffding_radius(n: np.ndarray | float, delta: float) -> np.ndarray | float:
26     """Two-sided Hoeffding confidence radius for variables in [0,1]."""
27     return np.sqrt(np.log(2.0 / delta) / (2.0 * np.asarray(n)))
28
29
30 def bernstein_radius_oracle(n: np.ndarray | float, delta: float, var: float) -> np.ndarray |
    float:
31     """A simple variance-aware Bernstein-style radius."""
32     x = np.log(2.0 / delta)
33     n = np.asarray(n)
34     return np.sqrt(2.0 * var * x / n) + x / (3.0 * n)
35
36
37 def fixed_time_coverage(seed: int = 0) -> pd.DataFrame:
38     rng = np.random.default_rng(seed)
39     p = 0.30
40     delta = 0.05
41     trials = 50_000
42     ns = np.array([10, 20, 50, 100, 200, 500, 1000])
```



```

43 rows = []
44 empirical = []
45 hoeffding_bounds = []
46 for n in ns:
47     samples = rng.binomial(1, p, size=(trials, n))
48     means = samples.mean(axis=1)
49     r = hoeffding_radius(n, delta)
50     miss = np.mean(np.abs(means - p) > r)
51     rows.append({
52         "experiment": "fixed_time_coverage",
53         "n": int(n),
54         "delta": delta,
55         "p": p,
56         "hoeffding_radius": float(r),
57         "empirical_miss_probability": float(miss),
58     })
59     empirical.append(miss)
60     hoeffding_bounds.append(delta)
61
62 fig, ax = plt.subplots(figsize=(6.5, 4.1))
63 ax.plot(ns, empirical, marker="o", label="Empirical miss probability")
64 ax.plot(ns, hoeffding_bounds, linestyle="--", label="Nominal delta = 0.05")
65 ax.set_xscale("log")
66 ax.set_xlabel("sample size n")
67 ax.set_ylabel("probability")
68 ax.set_title("Fixed-time Hoeffding intervals are conservative")
69 ax.grid(True, alpha=0.25)
70 ax.legend()
71 fig.tight_layout()
72 fig.savefig(OUT / "fixed_time_hoeffding.pdf")
73 fig.savefig(OUT / "fixed_time_hoeffding.png", dpi=220)
74 plt.close(fig)
75 return pd.DataFrame(rows)
76
77
78 def radius_width_plot() -> pd.DataFrame:
79     delta = 0.05
80     p = 0.05
81     var = p * (1.0 - p)
82     ns = np.arange(5, 1001)
83     h = hoeffding_radius(ns, delta)
84     b = bernstein_radius_oracle(ns, delta, var)
85
86     fig, ax = plt.subplots(figsize=(6.5, 4.1))
87     ax.plot(ns, h, label="Hoeffding radius")
88     ax.plot(ns, b, label="Bernstein radius using variance")
89     ax.set_xlabel("sample size n")
90     ax.set_ylabel("radius")

```

```

91     ax.set_title("Variance-aware error bars can be much shorter")
92     ax.grid(True, alpha=0.25)
93     ax.legend()
94     fig.tight_layout()
95     fig.savefig(OUT / "confidence_radius_widths.pdf")
96     fig.savefig(OUT / "confidence_radius_widths.png", dpi=220)
97     plt.close(fig)
98
99     return pd.DataFrame({
100         "experiment": "radius_widths",
101         "n": ns,
102         "delta": delta,
103         "p": p,
104         "variance": var,
105         "hoeffding_radius": h,
106         "bernstein_radius": b,
107     })
108
109
110 def run_bandit(policy: str, p: np.ndarray, T: int, rng: np.random.Generator) -> tuple[np.
    ndarray, np.ndarray]:
111     K = len(p)
112     counts = np.zeros(K, dtype=int)
113     sums = np.zeros(K, dtype=float)
114     sq_sums = np.zeros(K, dtype=float)
115     regret = np.zeros(T, dtype=float)
116     p_star = float(np.max(p))
117     cumulative_regret = 0.0
118
119     for t in range(min(K, T)):
120         a = t
121         r = rng.binomial(1, p[a])
122         counts[a] += 1
123         sums[a] += r
124         sq_sums[a] += r * r
125         cumulative_regret += p_star - p[a]
126         regret[t] = cumulative_regret
127
128     for t in range(K, T):
129         means = sums / np.maximum(counts, 1)
130         if policy == "Greedy":
131             a = int(np.argmax(means))
132         elif policy == "UCB-Hoeffding":
133             bonus = np.sqrt(2.0 * np.log(t + 1.0) / counts)
134             a = int(np.argmax(means + bonus))
135         elif policy == "UCB-Bernstein":
136             variances = np.zeros(K)
137             for i in range(K):

```

```

138         n = counts[i]
139         if n <= 1:
140             variances[i] = 0.25
141         else:
142             m = means[i]
143             variances[i] = max(0.0, sq_sums[i] / n - m * m)
144         log_term = np.log(t + 1.0)
145         bonus = np.sqrt(2.0 * variances * log_term / counts) + 3.0 * log_term / counts
146         a = int(np.argmax(means + bonus))
147     elif policy == "Thompson":
148         alpha = sums + 1.0
149         beta = counts - sums + 1.0
150         a = int(np.argmax(rng.beta(alpha, beta)))
151     else:
152         raise ValueError(f"Unknown policy: {policy}")
153
154     r = rng.binomial(1, p[a])
155     counts[a] += 1
156     sums[a] += r
157     sq_sums[a] += r * r
158     cumulative_regret += p_star - p[a]
159     regret[t] = cumulative_regret
160
161     return regret, counts
162
163
164 def bandit_experiment(seed: int = 7) -> pd.DataFrame:
165     rng = np.random.default_rng(seed)
166     p = np.array([0.01, 0.03, 0.05, 0.07])
167     T = 3000
168     trials = 150
169     policies = ["Greedy", "UCB-Hoeffding", "UCB-Bernstein", "Thompson"]
170     mean_regrets = {}
171     rows = []
172
173     for policy in policies:
174         regrets = []
175         counts_all = []
176         for _ in range(trials):
177             regret, counts = run_bandit(policy, p, T, rng)
178             regrets.append(regret)
179             counts_all.append(counts)
180         regrets = np.asarray(regrets)
181         counts_all = np.asarray(counts_all)
182         mean_regrets[policy] = regrets.mean(axis=0)
183         row = {"experiment": "bandit_regret", "policy": policy, "T": T, "trials": trials,
184               "final_regret": float(mean_regrets[policy][-1])}
185         for i, val in enumerate(counts_all.mean(axis=0)):

```

```

186         row[f"mean_pulls_arm_{i}"] = float(val)
187     rows.append(row)
188
189     fig, ax = plt.subplots(figsize=(6.7, 4.3))
190     x = np.arange(1, T + 1)
191     for policy in policies:
192         ax.plot(x, mean_regrets[policy], label=policy)
193     ax.set_xlabel("round t")
194     ax.set_ylabel("average cumulative regret")
195     ax.set_title("Concentration turns uncertainty into action")
196     ax.grid(True, alpha=0.25)
197     ax.legend()
198     fig.tight_layout()
199     fig.savefig(OUT / "ucb_regret_curves.pdf")
200     fig.savefig(OUT / "ucb_regret_curves.png", dpi=220)
201     plt.close(fig)
202
203     return pd.DataFrame(rows)
204
205
206 def main() -> None:
207     tables = [fixed_time_coverage(), radius_width_plot(), bandit_experiment()]
208     out = pd.concat(tables, ignore_index=True, sort=False)
209     out.to_csv(OUT / "concentration_results.csv", index=False)
210     print(out.tail(12).to_string(index=False))
211
212
213 if __name__ == "__main__":
214     main()

```

References

- [1] Jean-Yves Audibert, Rémi Munos, and Csaba Szepesvári. “Exploration-exploitation tradeoff using variance estimates in multi-armed bandits”. In: *Theoretical Computer Science* 410.19 (2009), pp. 1876–1902.
- [2] P. Auer, N. Cesa-Bianchi, and P. Fischer. “Finite-time analysis of the multiarmed bandit problem”. In: *Machine Learning* 47 (2002), pp. 235–256.
- [3] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013.
- [4] S. Bubeck and N. Cesa-Bianchi. “Regret analysis of stochastic and nonstochastic multi-armed bandit problems”. In: *Foundations and Trends in Machine Learning* 5.1 (2012), pp. 1–122.
- [5] T. Lattimore and C. Szepesvari. *Bandit Algorithms*. Cambridge University Press, 2020.